OmniStep 2.0 – Developer documentation

1 Unity project

The tutorial virtual reality (VR) application "OmniStep" (hereinafter "tutorial") was created with Unity 2022.3.18f1. Generally, LTS builds offer more stability and longer support than regular builds, making them suitable for projects that require a reliable development environment.

1.1 VR template

The Unity project was initiated using the VR template that comes preconfigured with essential settings targeted specifically for virtual reality. The template incorporates Unity's XR Interaction Toolkit 2.5.2, a high-level interaction system designed for creating VR and AR experiences. It offers cross-platform XR controller input via several supported provider plugins, such as Oculus XR or OpenXR. In order to ensure compatibility with a wide range of VR headsets, the OpenXR Plugin 1.10.0 was chosen. Unlike the Oculus XR Plugin, which only supports Oculus headsets, OpenXR offers support for Meta headsets, HTC Vive headsets, Valve SteamVR, and other.

XR Origin

The *XR Origin*, provided in the XR Interaction Toolkit, represents the main VR component in a properly set up scene for virtual reality. It serves as the starting point for configuring the user's interactions with the virtual world, ensuring proper tracking and movement within the virtual space.

VR Simulator

To be able to walk through the tutorial when a VR headset is not connected, the application switches to a simulator mode that can be controlled using keyboard and mouse. This is enabled via the XR Device Simulator provided by Unity. However, it is intended *strictly for debugging purposes* and should not be considered a substitute for actual VR gameplay.

1.2 KAT Walk integration

In order to make the application compatible with the KAT Walk treadmills, the KAT Unity integration SDK 1.0 was used. It is the official SDK development tool based on Unity XR, developed by KAT VR, and provides compatibility with the following devices: KAT Walk mini S, KAT Loco S, KAT Walk C, KAT Walk C2. Implementing the support requires the use of the *KATXR Walker* script, which must be added to the GameObject with the *XR Origin* present.

1.3 Folder structure

The project mostly follows a standard Unity folder structure, which includes the *Assets* folder as the main directory for all project content. Within it, subfolders are organized by purpose: *Scenes* for scene files, *Scripts* for C# code, *Prefabs* for reusable objects, *Materials* for material assets, *Models* for 3D assets, and *Animations* for animation assets. Additional folders can be found in the project, such as *GeneratedTTS* for files related to the custom **TTS Localization Kit** package for text-to-speech. This structure ensures the project remains clean, organized, and easy to navigate.

1.4 Scene hierarchy

The whole tutorial is contained in a single scene called "TutorialScene". As shown in Figure 1, it is organized into several sections for better structure and navigation. The Complete XR Origin Set Up contains all functionality necessary for a VR headset to operate properly.

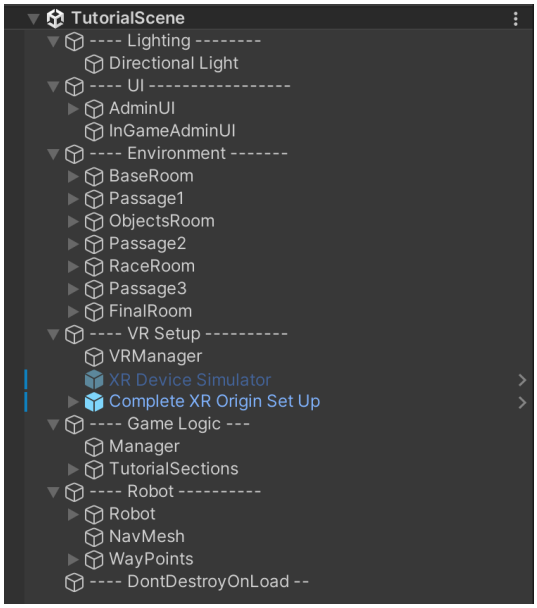

Figure 1. Tutorial scene hierarchy.

2 Localization package and package for text-to-speech

The application offers localization in English and Czech language. In text form, this is easily implemented using Unity's Localization package. This involves creating a table in Unity, where localized text entries are represented by a string key. For each locale, the translation is filled in the table's row belonging to a specific key. This offers an easily extensible solution when more languages are needed.

However, the tutorial provides instructions in both text and verbal form. While the package also supports the use of localized assets such as audio, the process of managing these assets introduces extra work. The main problem is that the audio needs to be imported into Unity and manually assigned in the asset localization table. This becomes quite tedious once a larger number of entries is needed. Additionally, any change in the localized text itself means the corresponding audio clip needs to be regenerated.

In order to improve this process and allow for more dynamic editing, a custom TTS Localization Kit package for text-to-speech was developed for the tutorial to automate this process. The solution reduces manual workload and allows for quick audio regeneration via a simple button click. It is particularly useful for the management of the instructions presented by the robot NPC (non-player character) in the tutorial. The following subsections describe the configuration and implementation of the key scripts.

Configuration

The custom TTS Localization Kit package for text-to-speech can be added to a Unity project via the package manager using the GitHub URL. The package relies on AI generation and integrates Google Text-to-Speech API (TTS API). Special Access credentials in the form of a JSON file are needed and can be obtained from Google Cloud console as described in the above-linked documentation. After that, the *TTSGeneratorSettings* script must be placed in the Unity scene and filled with the path to the credentials file.

TextAndAudioManager<T>

The *TextAndAudioManager<T>* generic class serves as a base for the use of the abilities of the package in the project. One such example in this project would be the robot NPC. *RobotTextAndAudioManager* inherits from the base class, supplying itself as the T parameter. This ensures there can be only one *RobotTextAndAudioManager* in the scene, as the implementation follows the singleton pattern. The script is placed on the robot object in the scene, and if correctly set up as displayed in Figure 2, the audio corresponding to the string localization table will be regenerated on button click.

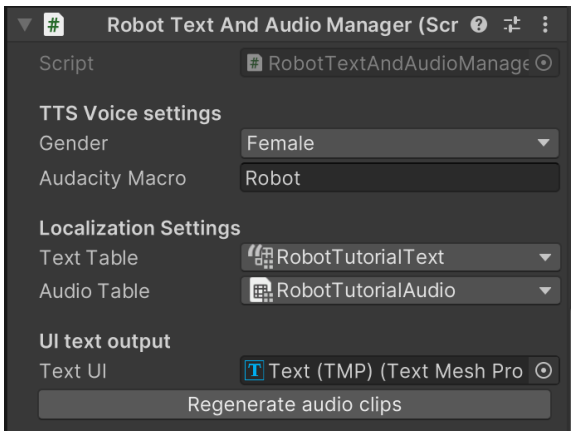

Figure 2. "RobotTextAndAudioManager" setup in the tutorial scene.

TTSGenerator

TTSGenerator is a C# script that facilitates the whole automation process. Once the *Regerate audio clips* button is pressed inside the Unity editor, the asynchronous *RegenrateAudio* function is called. It receives

the references to appropriate localization tables, path to the JSON key for Google TTS API, gender of the resulting voice, and an optional Audacity Macro name if any post-processing is desired.

On the first run, the function exports all of the localized text into JSON format, as shown in Figure 3. On further use, only entries of edited text are updated, marking the field *Regenerate* as true. In this way, only the changes are regenerated and not the whole table.


```
{
  "entries": [
    {
      "key": "good-job",
      "texts": {
        "en": {
          "value": "Good job.",
          "regenerate": true
        },
        "cs": {
          "value": "Dobrá práce.",
          "regenerate": true
        }
      }
    },
    {
      "key": "successful-finish",
      "texts": {
        "en": {
          "value": "You successfully finished the training.",
          "regenerate": true
        },
        "cs": {
          "value": "Uspěšně jste splnili trénink.",
          "regenerate": true
        }
      }
    }
  ]
}
```


Figure 3. Structure of the JSON file for audio generation.

Once the JSON file is up to date, the *google_tts* Python script is called, generating all the audio files. The function awaits completion of the generation task and imports all of the generated audio into the localization asset table under a key that matches the key of the text entry in the string localization table.

google_tts

The *google_tts* Python script deserializes the previously mentioned JSON file with the texts. It loops through all the entries, and for those marked for regeneration, it calls the Google TTS API with appropriate settings such as language and gender. Additionally, if an Audacity Macro is specified, it applies it via the Audacity Scripting utilizing named pipes. For this step to be successful, Audacity must be running on the device and have a macro with the corresponding name configured. If that is not the case, the whole generation results in an error that is then printed to the Unity debug console.

3 Hand poser

To allow for custom hand grab poses for interactable objects in the tutorial, a hand poser tool was implemented. It is an editor tool that allows for the creation of visual hand poses. The pose is created in edit mode via joint manipulation and saved to the objects, as shown in Figure 4. When the object is grabbed in the scene during runtime, if a custom set pose is found on the intractable object, it is applied to the hand model. Examples of the custom poses in the tutorial are displayed in Figure 5.

The management of this process is handled via a *HandPoserManager* editor script. It provides functionality to create, update, and manipulate poses for both left and right hand.

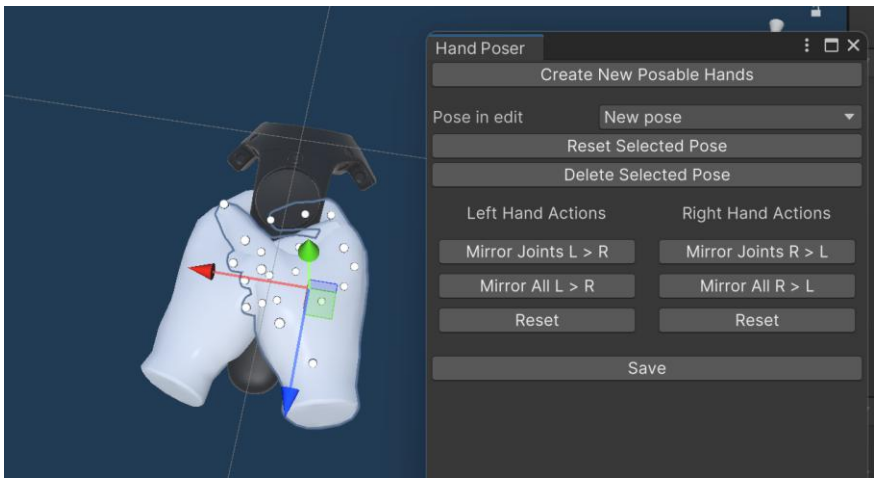

Figure 4. Showcase of the hand poser in action.

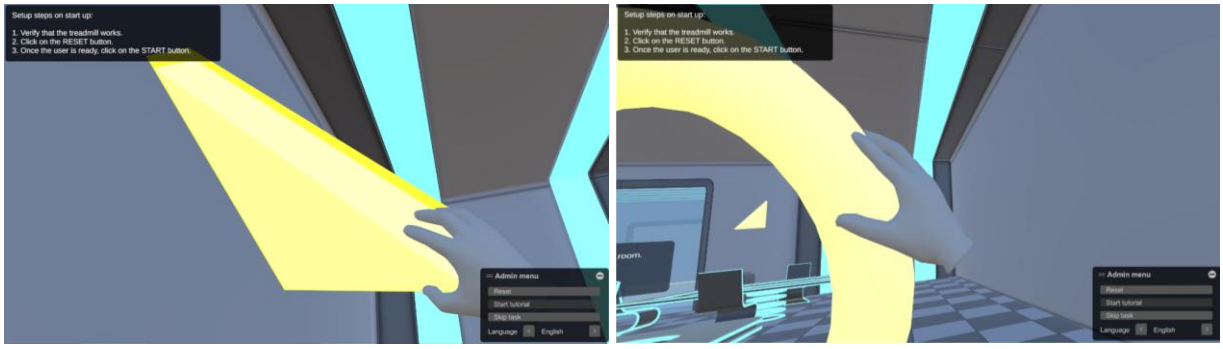

Figure 5. Custom hand poses for different objects.

4 Tutorial flow implementation

The tutorial flow implementation is based on individual section and task components. However, the division is slightly more granular. The modular implementation architecture ensures flexibility, as sections and tasks can be, to some extent, rearranged, reused, or updated independently, making it easier to iterate on the tutorial content.

The whole tutorial loop operates on an asynchronous basis, passing on cancellation tokens between the components. This enables a skip task functionality based on the use of cancellation tokens. However, it is mainly meant for debugging purposes.

Coroutines versus async/await

In Unity, both **coroutines** and **async/await** are used to handle asynchronous tasks, but serve different purposes and have distinct use cases.

Coroutines are a Unity-specific feature, designed to perform tasks over multiple frames without blocking the main thread. This makes them ideal for managing game-specific tasks such as timed actions, animations, or waiting for events. However, they are not able to return values and do not implement a simple cancellation mechanism like async/await.

On the other hand, **async/await** is a general-purpose asynchronous programming model in C#. Usually, they are used to handle long-running background tasks. However, in this case, the main motivation for their use was the simplicity of cancellation, easier implementation of task awaiting inside the tasks, and the ability to return values. As Unity does not handle async/await as safely as it handles coroutines, developers must ensure that Unity API calls are executed on the main thread and properly handled via the use of cancellation tokens.

TutorialSectionsManager class

The *TutorialSectionsManager* script represents the main tutorial loop. It runs a section and awaits its completion before moving to the next one. If the application is closed, the cancellation token ensures that the process is ended properly. Figure 6 shows the script's setup within the Unity editor, presenting all sections of the tutorial. Only one instance of this script must be present in the scene. It is implemented using the singleton pattern, resulting in an error when more than one instance is present. In addition to managing the tutorial flow, the script collects analytics data from each section and saves it to a file at the end of the tutorial. The data contains the time duration it took for the user to complete each task.

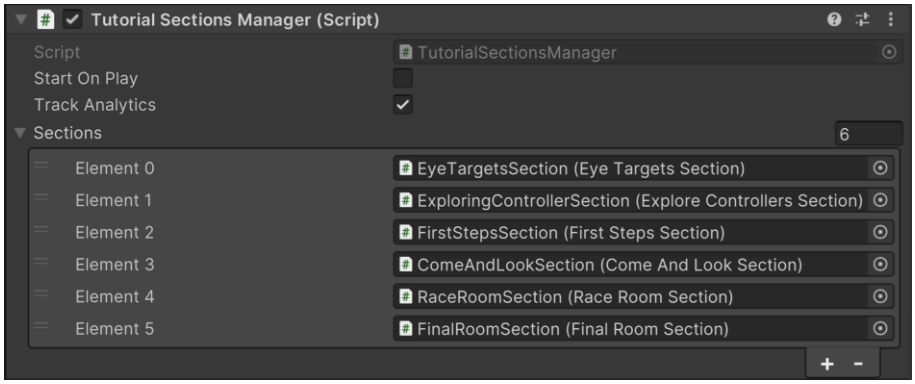

Figure 6. "TutorialSectionsManager" setup in the tutorial.

SectionBase class

SectionBase is a base class for the individual sections within the tutorial. In the current implementation, this component is primarily used for the logical organization of tasks and measuring the time of their execution. Additionally, it also propagates proper robot NPC position between tasks, ensuring that on invoking the skip task, the robot NPC ends up in the desired position.

SectionTaskBase class

The *SectionTaskBase* represents a base class that all of the tasks inherit from. In the current implementation, a task generally represents a logic encapsulation for one assignment for the user. This component provides access to common components, functionality, and handles the cancellation token, ensuring tasks can be interrupted gracefully when the skip task is invoked or the application closes.

CleanupOnSkip is a virtual method that handles the task cleanup when it is skipped. In order to ensure proper continuity of the tutorial, if a task is skipped, this method applies all the changes as if it was completed. For example, if during a task an interactable element should appear and stay in the scene, the same must happen even if the task is skipped. It is up to the implementation of the specific task to override this method and apply the necessary changes.

Extendibility

Thanks to the modular architecture of the tutorial, adding new tasks and sections should be relatively simple. New components must derive from the aforementioned base classes and be added inside the editor to an appropriate place in a section or the *TutorialSectionsManager* itself. However, it is up to the developer to ensure that proper continuity of the tutorial is preserved. Due to the mostly linear nature of the tutorial, some tasks must precede other for the tutorial to work correctly. For example, the task asking the user to throw a cube cannot be placed before the task where the cube appears and the user learns how to interact with it. However, it is entirely valid to insert a new task between these tasks and ask the user to interact with the cube in some new additional way.

5 Software requirements and installation

When running the application, Windows 10 and Windows 11 are the recommended operating systems. It can be started via the "OmniStep.exe" present in the "Builds/OmniStep_2.0" folder. In order to run the application in VR mode, SteamVR has to be installed, and one of the supported VR headsets must be connected. The application has been tested with HTC Vive Pro and HTC Vive Pro 2, but is expected to work with other OpenXR-compatible headsets as well.

The application supports the use of the KAT Walk mini S and KAT Walk C omnidirectional treadmills. Each requires specialized software to be installed – KAT Industry for KAT Walk mini S and KAT Gateway for KAT Walk C. Having both of these installed on one device seems to cause issues with treadmill input; therefore, it is highly discouraged.

5.1 Running application with KAT Walk mini S

When running the application with KAT Walk mini S, it must be realized through KAT Industry software, as shown in the document "OmniStep 2.0 – How to launch with KAT Industry 3.1.9.pdf" in the Documentation folder.

5.2 Running application with KAT Walk C

With KAT Walk C, KAT Gateway software has to be running, but the application itself is run separately, as shown in the document "OmniStep 2.0 – How to launch with KAT Gateway 2.4.2.pdf" in the Documentation folder.

6 Assets used

In order to create the desired visuals, free asset packs were used for the environment and robot NPC. Several of these assets were modified in Blender to achieve the desired final look presented in the tutorial. The list of the assets used is as follows:

- Asset 1: *Sci-Fi table*. (2024). https://sketchfab.com/3d-models/sci-fi-table-298aa4e4148f4d3b95c6d35efecd52b1
- Asset 2: *Loot Box*. (2020). https://sketchfab.com/3d-models/loot-box-24d1d9be93954d3eb7807f8b528d6d98
- Asset 3: *Jammo Character*. (2020). https://assetstore.unity.com/packages/3d/characters/jammo-character-mix-and-jam-158456
- Asset 4: *3D Free Modular Kit*. (2024). https://assetstore.unity.com/packages/3d/environments/3d-free-modular-kit-85732
- Asset 5: *Star*. (2019). https://sketchfab.com/3d-models/star-5b21e84ddc0342359cd272a9c6dd31cb
- Asset 6: *Sci-Fi Target*. (2023). https://sketchfab.com/3d-models/sci-fi-target-cfd36c52c80b4f4ca8d69cec1cf7d293
- Asset 7: *success_bell*. (2020). https://freesound.org/people/MLaudio/sounds/511484/

7 Acknowledgment

This software documentation is partly based on and includes material originally presented in the master's thesis by Eliška Suchardová, supervised by Michal Sedlák, at Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic. The original text has been edited, updated, and supplemented for the current software documentation. This work was supported by the Center for Virtual Reality Research in Mental Health and Neuroscience (Center for VR Research) at National Institute of Mental Health in the Czech Republic (NIMH CZ).